EC-110: Computing Fundamentals

Lecture no 10: Basic Elements of C++

The Basics of a C++ Program

- Function: collection of statements; when executed, accomplishes something.
  - May be predefined or standard
- Syntax: rules that specify which statements (instructions) are legal.
- Programming language: a set of rules, symbols, and special words.
- Semantic rule: meaning of the statements/instructions.

Comments

- Comments are for the reader, not the compiler
- Two types:
  - Single line

```
// This is a C++ program. It prints the sentence:
// Welcome to C++ Programming.
```

  - Multiple line

```
/*
   You can include comments that can
   occupy several lines.
*/
```

Special Symbols

- Special symbols

+	?
–	,
*	<=
/	!=
.	==
;	>=

Reserved Words (Keywords)

- Reserved words, keywords, or word symbols
  - Include:
    - `int`
    - `float`
    - `double`
    - `char`
    - `const`
    - `void`
    - `return`

Identifiers

- Consist of letters, digits, and the underscore character (_)
- *Must begin with a letter or underscore.*
- C++ is case sensitive
  - `NUMBER` is not the same as `number`
- Two predefined identifiers are `cout` and `cin`
- Unlike reserved words, predefined identifiers may be redefined, but it is not a good idea

Identifiers (continued)

- The following are legal identifiers in C++:
  - `first`
  - `conversion`
  - `payRate`

TABLE 2-1 Examples of Illegal Identifiers

Illegal Identifier	Description
`employee Salary`	There can be no space between `employee` and `Salary`.
`Hello!`	The exclamation mark cannot be used in an identifier.
`one+two`	The symbol + cannot be used in an identifier.
`2nd`	An identifier cannot begin with a digit.

Whitespaces

- Every C++ program contains whitespaces
  - White spaces include blanks, tabs, and newline characters
- Used to separate special symbols, reserved words, and identifiers

Data Types

- C++ data types fall into three categories:

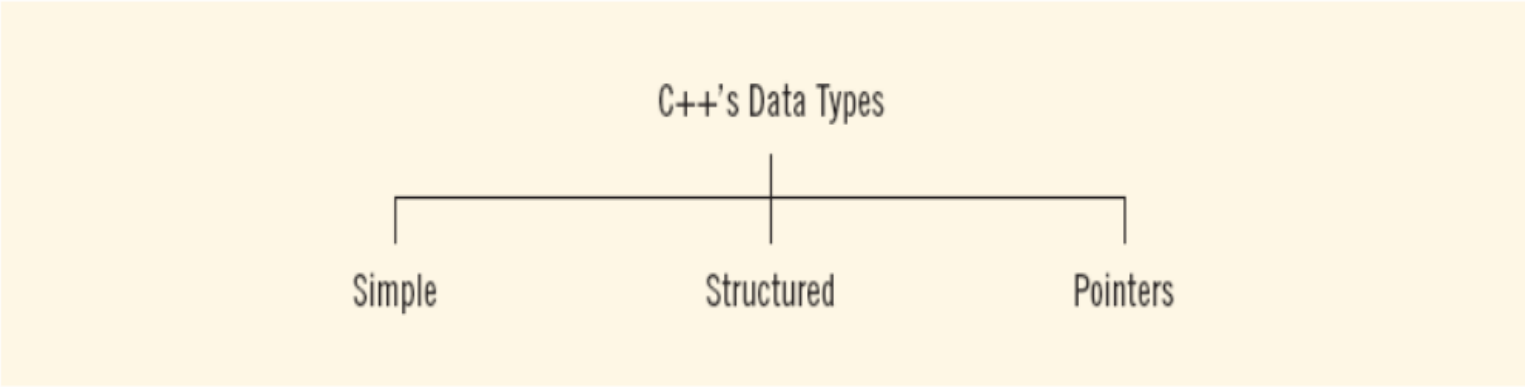


FIGURE 2-1 C++ data types

Simple Data Types

- Three categories of simple data
  - Integral: integers (numbers without a decimal)
  - Floating-point: decimal numbers
  - Enumeration type: user-defined data type

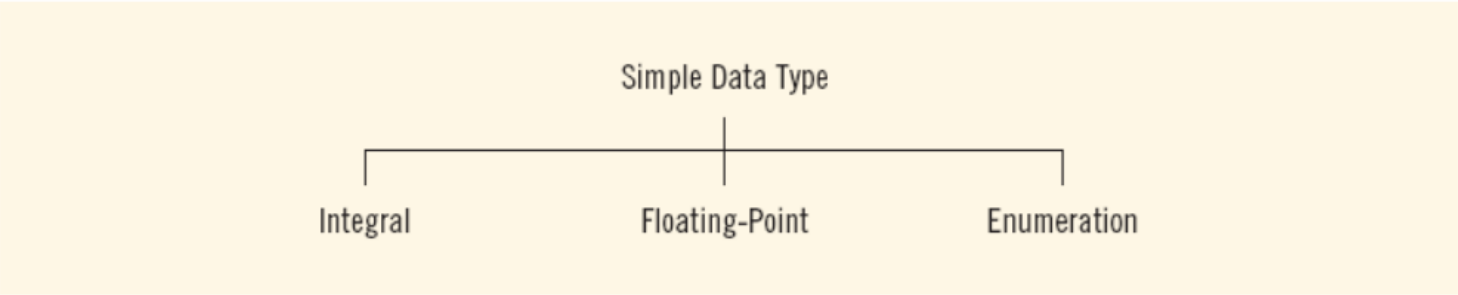


FIGURE 2-2 Simple data types

Simple Data Types (continued)

- Integral data types are further classified into nine categories:

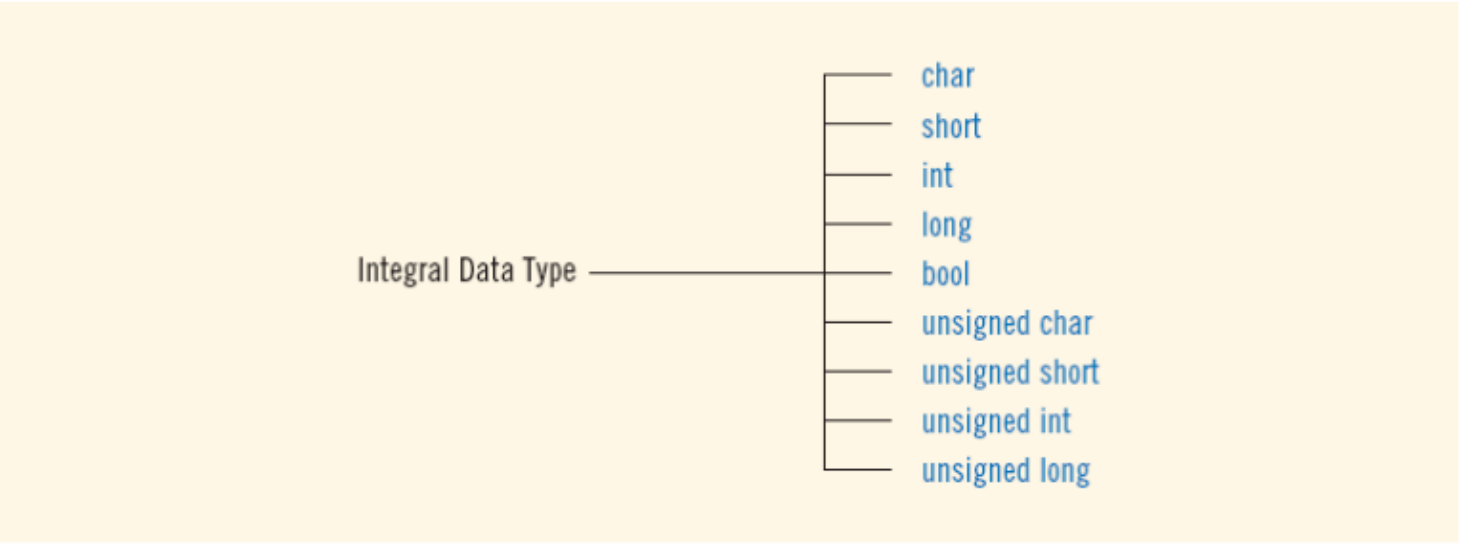


FIGURE 2-3 Integral data types

Simple Data Types (continued)

TABLE 2-2 Values and Memory Allocation for Three Simple Data Types

Data Type	Values	Storage (in bytes)
`int`	-2147483648 to 2147483647	4
`bool`	`true` and `false`	1
`char`	-128 to 127	1

- Different compilers may allow different ranges of values

`int` Data Type

- Examples:

```
-6728
0
78
+763
```

- Positive integers do not need a `+` sign.

`bool` Data Type

- `bool` type
  - Two values: `true` and `false`
  - Manipulate logical (Boolean) expressions
- `true` and `false` are called logical values
- `bool`, `true`, and `false` are reserved words

`char` Data Type

- The smallest integral data type
- Used for characters: letters, digits, and special symbols
- Each character is enclosed in single quotes
  - `'A'`, `'a'`, `'0'`, `'*'`, `'+'`, `'$'`, `'&'`
- A blank space is a character and is written `' '`, with a space left between the single quotes.

Floating-Point Data Types

- C++ uses scientific notation to represent real numbers (floating-point notation)

TABLE 2-3 Examples of Real Numbers Printed in C++ Floating-Point Notation

Real Number	C++ Floating-Point Notation
75.924	7.592400E1
0.18	1.800000E-1
0.0000453	4.530000E-5
-1.482	-1.482000E0
7800.0	7.800000E3

Floating-Point Data Types (continued)

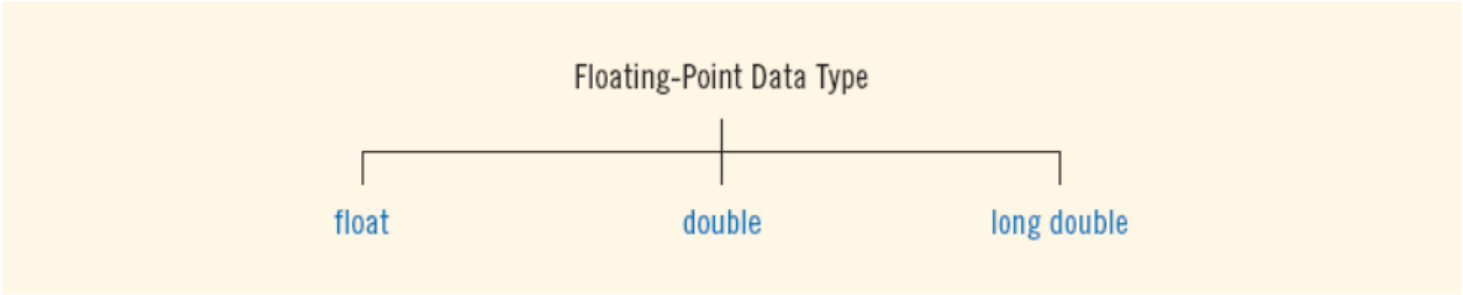


FIGURE 2-4 Floating-point data types

- `float`: represents any real number
  - Range: (four bytes)
- `double`: represents any real number
  - Range: (eight bytes)
- On most newer compilers, data types `double` and `long double` are same

Arithmetic Operators and Operator Precedence

- C++ arithmetic operators:
  - + addition
  - - subtraction
  - * multiplication
  - / division
  - % modulus operator
- +, -, *, and / can be used with integral and floating-point data types

Order of Precedence

Complex logical expressions can be difficult to evaluate like: 11 > 5 || 6< 15 && 7 >=8

To work with complex logical expressions, there must be some priority scheme for evaluating operators. Table next slide shows the order of precedence of some C++ operators including the arithmetic, relational, and logical operators.

Order of Precedence (continued)

Operators	Precedence
!, +, – (unary operators)	first
*, /, %	second
+, –	third
<, <=, >=, >	fourth
==, !=	fifth
&&	sixth
||	seventh
= (assignment operator)	last

Table 11

Order of Precedence (continued)

You can insert parentheses into an expression to clarify its meaning.

You can also use parentheses to nullify the precedence of operators.

Using the standard order of precedence, the expression:

11 > 5 || 6< 15 && 7 >=8

is equivalent to:

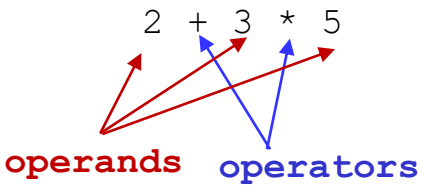
11 > 5 || (6< 15 && 7 >=8)

Order of Precedence (continued)

In this expression, `11 > 5` is `true`, `6 < 15` is `true`, and `7 >= 8` is `false`. Substitute these values in the expression `11 > 5 || (6 < 15 && 7 >= 8)` to get `true || (true && false )` = `true || false` = `true`. Therefore, the expression `11 > 5 || (6 < 15 && 7 >= 8)` evaluates to `true`.

Expressions

- If all operands are integers
  - Expression is called an integral expression
    - Yields an integral result
    - Example: `2 + 3 * 5`



- If all operands are floating-point
  - Expression is called a floating-point expression
    - Yields a floating-point result
    - Example: `12.8 * 17.5 - 34.50`

Mixed Expressions

- Mixed expression:
  - Has operands of different data types
  - Contains integers and floating-point
- Examples of mixed expressions:

```
2 + 3.5
6  /  4 + 3.9
5.4  *  2 - 13.6 + 18  /  2
```

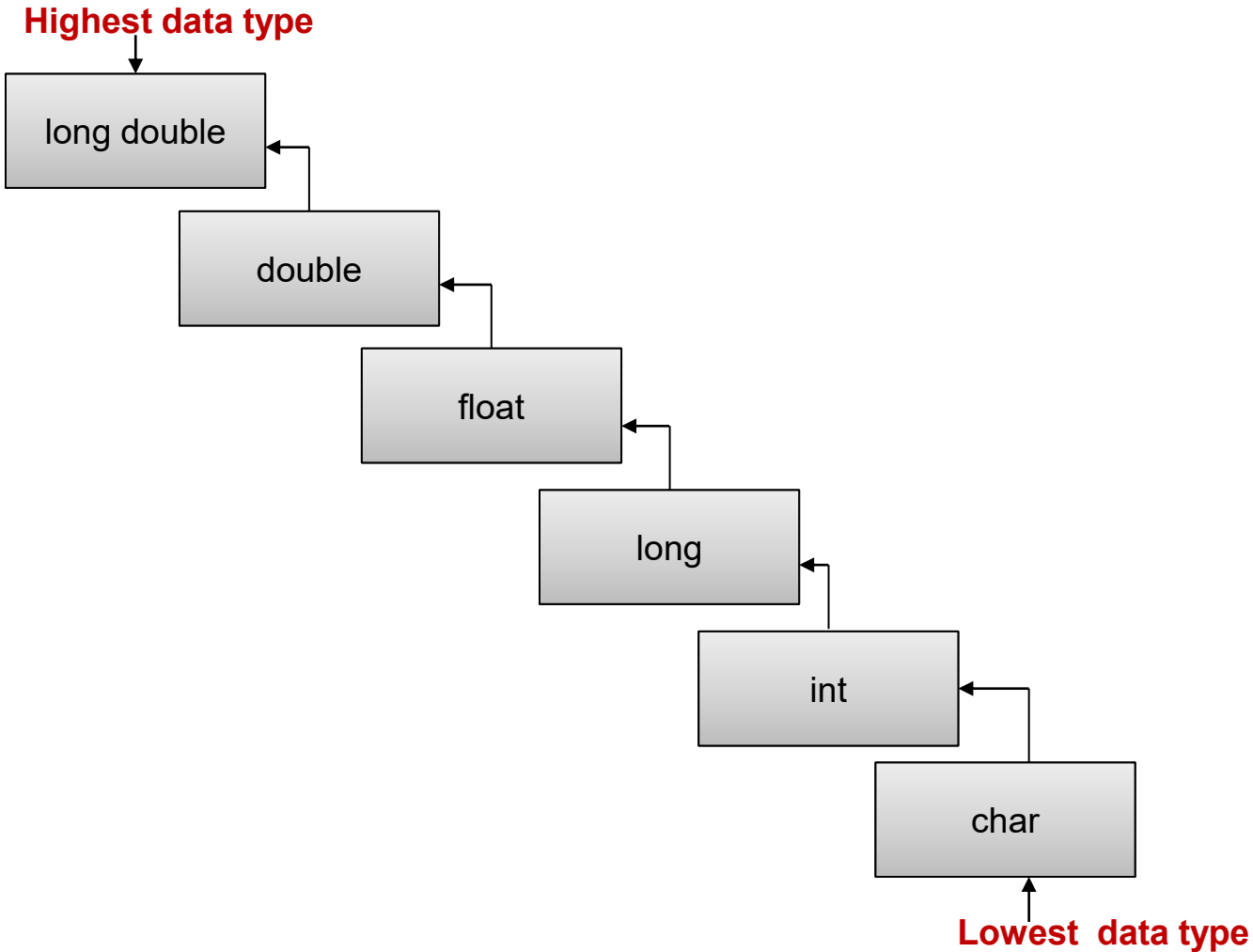
Type Conversion/Type Casting

- The process of converting the data type of a value during execution is know as ***type casting***.
- Type casting can be performed in two ways:
  - Implicit type casting
  - Explicit type casting

Implicit type casting is performed automatically by the C++ compiler. Some examples of implicit conversions are given on table next slide

Type Conversion/Type Casting

- Different types of variables are as follows:



Type Conversion/Type Casting

Expression	Intermediate Type
Char + float	Float
int - long	Long
int * double	double
float / long double	Long double

Type Conversion/Type Casting

Example: Suppose x is an integer and y is a long variable and the following expression is evaluated: ***x + y***.
In the above expression, data type of ***x*** is lower than data type of ***y***. So the value of ***x*** will be converted into long during the evaluation of expression. The data type of ***x*** is not changed. Only the data type of the value of ***x*** is changed during the evaluation of expression.

Type Conversion/Type Casting

- ***Explicit type casting:*** is performed by programmer. It is performed by using ***cast operator***.

Syntax: The syntax of using cast operator is as follows:

(type) expression;

type: It indicates the data type to which operand is to be converted.

expression: It indicates the constant, variable or expression whose data type is to converted. For example **(int) x;** // this statement will convert the value of **x** into integer.

String

A collection of characters written in double quotations is called ***string*** or ***string constant***. It may consist of any alphabetic characters, digits, and special symbols.

A string is stored as an array of characters. The array is terminated by a special symbol known as ***null character***. It is denoted by **\0** and is used to indicate the end of string. It consists of black slash and zero.

The values stores in an array of characters can be manipulated individually using the index of the array. The user can *input, process* and *displays* the individual characters of the string in the same way as an array. ***Examples:*** some examples of strings are as follows:

"Pakistan" , "123", "99-Mall Road, Lahore" .

Putting Data into Variables

- Ways to place data into a variable:
  - Use C++'s assignment statement
  - Use input (read) statements

Assignment Statement

- The assignment statement takes the form:

```
variable = expression;
```

- Expression is evaluated and its value is assigned to the variable on the left side
- In C++, = is called the assignment operator

Assignment Statement (continued)

EXAMPLE 2-13

```
int num1, num2;
double sale;
char first;
string str;

num1 = 4;
num2 = 4 * 5 - 11;
sale = 0.02 * 1000;
first = 'D';
str = "It is a sunny day.";
```

EXAMPLE 2-14

```
1.  num1 = 18;
2.  num1 = num1 + 27;
3.  num2 = num1;
4.  num3 = num2 / 5;
5.  num3 = num3 / 4;
```

Declaring & Initializing Variables

- Variables can be initialized at the time of declaration:

```
int first=13, second=10;
char ch=' ';
double x=12.6;
```

- All variables must be initialized before they are used.

Input (Read) Statement

- `cin` is used with `>>` to gather input
- The stream extraction operator is >>
- For example

```
cin >> var;
```

  - Causes computer to get a value from keyboard and store it in a variable ***var***.

Input (Read) Statement (continued)

- Using more than one variable in `cin` allows more than one value to be read at a time
- For example, if `feet` and `inches` are variables of type `int`, a statement such as:

```
cin >> feet >> inches;
```

  - Inputs two integers from the keyboard
  - Places them in variables `feet` and `inches` respectively

Input (Read) Statement (continued)

EXAMPLE 2-17

```
#include <iostream>

using namespace std;

int main()
{
    int feet;
    int inches;

    cout << "Enter two integers separated by spaces: ";
    cin >> feet >> inches;
    cout << endl;

    cout << "Feet = " << feet << endl;
    cout << "Inches = " << inches << endl;

    return 0;
}
```

Sample Run: (In this sample run, the user input is shaded.)

```
Enter two integers separated by spaces: 23 7

Feet = 23
Inches = 7
```

Variable Initialization

- There are two ways to initialize a variable:

```
int feet;
```

  - By using the *assignment statement*

```
feet = 35;
```

  - By using a *read statement*

```
cin >> feet;
```

Increment & Decrement Operators

Increment operator

- The increment operator is used to increase the value of a variable by 1. it is denoted by the symbol **++**. It is a unary operator and works with single variable.
- The increment operator cannot increment the value of constants and expressions. For example, **A++** and **X++** are valid statements but 10++ is an invalid statement. Similarly, **(a+b)++** or **++(a+b)** are also invalid.

Increment & Decrement Operators (continued)

Increment operator

Increment operator can be used in two forms:

i. ***Prefix Form-*** In prefix form, the increment operator is written **before** the variable as follows: **++y;** // this statement increments the value of variable **y** by 1.

ii. ***Postfix Form-*** In postfix form, the increment operator is written **after** the variable as follows: **y++;** // the statement increments the value of **y** by 1.

Increment & Decrement Operators (continued)

Difference between Prefix and Post Increment operator

- When increment operator *is used independently*, prefix and postfix work similarly. For example, the result of A++ and ++A is same.
- When increment operator *is used in a larger expression with other operators*, prefix and post fix forms work differently. For example, the results of two statements **A = ++B** and **A = B++** are different.

Increment & Decrement Operators (continued)

Difference between Prefix and Post Increment operator

In prefix form, the statement **A = ++B** contains two operators ++ and =. It works in following order:

1. It increments the value of **B** by 1.
2. It assigns the value of **B** to **A**.

The above statement is equivalent to the following two statements:

```
++B;
A = B;
```

Increment & Decrement Operators (continued)

Difference between Prefix and Post Increment operator

In postfix form the statement **A = B++** works in the following order:

1. It assigns the value of **B** to **A**.
2. It increments the value of **B** by 1.

The above statement is equivalent to the following two statements:

```
A = B;
B++;
```

Increment & Decrement Operators

Decrement operator

- The decrement operator is used to decrease the value of a variable by 1. it is denoted by the symbol **--**.It is a unary operator and works with single variable.
- The decrement operator cannot decrement the value of constants and expressions. For example, **A--** and **X--** are valid statements but 10-- is an invalid statement. Similarly, **(a+b)--** or **--(a+b)** are also invalid.

Increment & Decrement Operators (continued)

Decrement operator

Decrement operator can be used in two forms:

i. ***Prefix Form-*** In prefix form, the decrement operator is written **before** the variable as follows: **--y;** // this statement decrements the value of variable **y** by 1.

ii. ***Postfix Form-*** In postfix form, the decrement operator is written **after** the variable as follows: **y--;** // the statement decrements the value of **y** by 1.

Increment & Decrement Operators (continued)

Difference between Prefix and Post Decrement operator

In prefix form the statement **A = --B** contains two operators -- and =. It works in following order:

1. It decrements the value of **B** by 1.
2. It assigns the value of **B** to **A**.

The above statement is equivalent to the following two statements:

```
--B;
A = B;
```

Increment & Decrement Operators (continued)

Difference between Prefix and Post Decrement operator

In postfix form the statement **A = B--** works in the following order:

1. It assigns the value of **B** to **A**.
2. It decrements the value of **B** by 1.

The above statement is equivalent to the following two statements:

```
A = B;
B--;
```

Output

- The process of getting something out from computer is called **output**.
- The **cout** object is used with insertion operator (<<).
- The syntax of using `cout` object is as follows:

```
cout<<variable/constant/expression;
```

- The message or the value of variable followed by the insertion operator is displayed on the screen.

Output (continued)

- Example: `endl` causes insertion point to move to beginning of next line

EXAMPLE 2-21

	Statement	Output
1	`cout << 29 / 4 << endl;`	`7`
2	`cout << "Hello there." << endl;`	`Hello there.`
3	`cout << 12 << endl;`	`12`
4	`cout << "4 + 7" << endl;`	`4 + 7`
5	`cout << 4 + 7 << endl;`	`11`
6	`cout << 'A' << endl;`	`A`
7	`cout << "4 + 7 = " << 4 + 7 << endl;`	`4 + 7 = 11`
8	`cout << 2 + 3 * 5 << endl;`	`17`
9	`cout << "Hello \nthere." << endl;`	`Hello` `there.`

Output (continued)

TABLE 2-4 Commonly Used Escape Sequences

	Escape Sequence	Description
`\n`	Newline	Cursor moves to the beginning of the next line
`\t`	Tab	Cursor moves to the next tab stop
`\b`	Backspace	Cursor moves one space to the left
`\r`	Return	Cursor moves to the beginning of the current line (not the next line)
`\\`	Backslash	Backslash is printed
`\'`	Single quotation	Single quotation mark is printed
`\"`	Double quotation	Double quotation mark is printed

Preprocessor Directives

- ***Preprocessor directive*** is an instruction given to the compiler before the execution of actual program. Preprocessor directive is also know as **compiler directive**.
- The preprocessor directives start with hash symbol #. These directives are written at the start of the program.
- ***Include preprocessor*** directive is used to include header files in the program. The syntax of using this directive is as follows: *#include <iostream.h> // this statement tells the compiler to include the file **iostream.h** in source program before compiling it*.
- To use the `string` type, you need to access its definition from the header file `string`
- Include the following preprocessor directive:

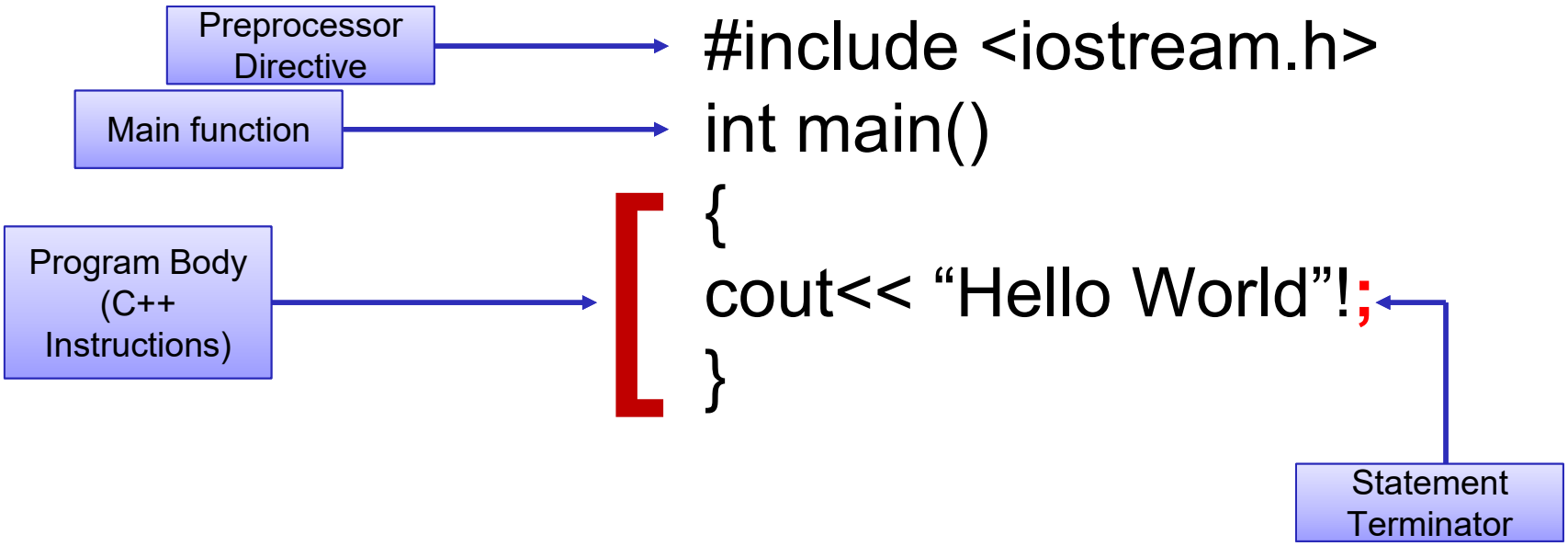
```
#include  <string>
```

Preprocessor Directives (continued)

- ***Header Files*** are collection of standard library functions to perform different tasks. There are many header files for different purposes. Each header file contains different types of predefined functions. The extension of a header file is **.h** These files are provided by C++ compiler system. The header files are normally stored in **INCLUDE** subdirectory. The syntax of using header files is as follows: *#include<header_file_name>*
- ***main( ) Function*** is the starting point of C++ program. When the program is run the control enters main() function and starts executing its statements. The syntax is :

```
int main()
{
Body of the main function (C++ statements)
}
```

Basic Structure of a C++ Program



Algorithm/Flowchart and Code for a C++ Program

Program: *Write a program that inputs base and height from the user and calculates area of a triangle by using the formula Area = ½(Base*Height).*

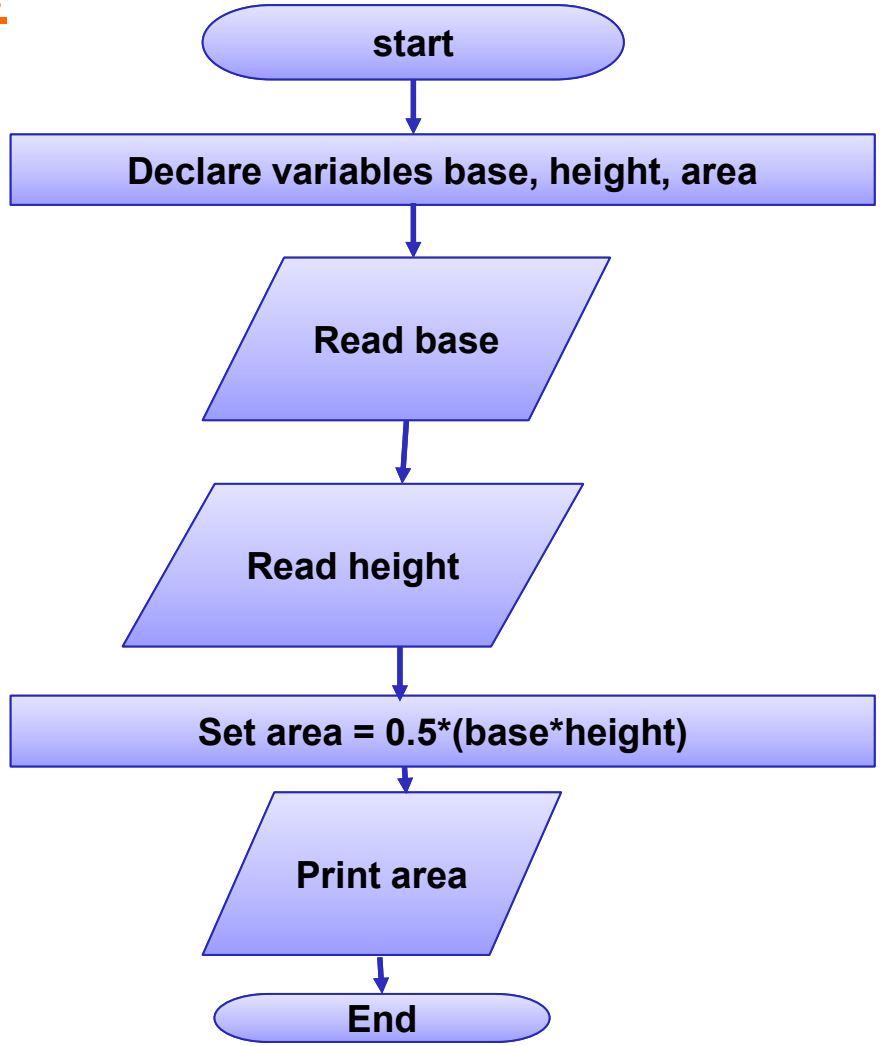
1. Write down *an algorithm* for the above program.
2. Draw *a flowchart* for the above program.
3. Write down *the program (running code)* for the above program.

Algorithm:

1.Start.
2.Input base and height.
3.Calculate area = 0.5*(base*height).
4.Print "Area of Triangle = ", area.
5.End.

Algorithm/Flowchart and Code for a C++ Program (continued)

Flowchart:



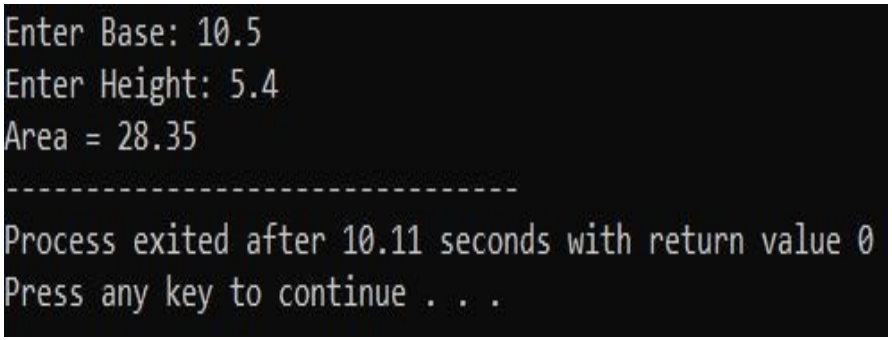
Algorithm/Flowchart and Code for a C++ Program

Program code:

```
#include<iostream>
using namespace std;

int main()
{
        float base, height;
        double area;
        cout<<"Enter Base: ";
        cin>>base;
        cout<<"Enter Height: ";
        cin>>height;
        area = 0.5*(base*height);
        cout<<"Area = "<<area;
        return 0;
}
```

OUTPUT



References

1. *"C++ Programming: From Problem Analysis to Program Design",6/ed by D.S. Malik, (2013), CENGAGE Learning.*
2. *"Object Oriented Programming in C++," 4/ed by Robert Lafore (2001) .*
3. *"C++ How to Program," 5/ed by Deitel & Deitel (2005).*
4. *"Program with C++ Object Oriented Programming Aikman Series", by C.M Aslam and T.A Qureshi.*
5. *Related documents from open source, mainly internet.*